

Orchestrate data processing in Spark using AWS Step Functions

SPL-BE-200-ANDEB2-1 - Version 1.0.4

Lab overview

You have completed your research on how to use Amazon EMR with Apache Spark by building your prototype stock application in an Amazon EMR notebook (based on Jupyter Notebook). Now, you have been asked to automate the batch processing job for 2020 stock data and work on reducing costs.

Your task is to run batch processing on 2020 stock data and append it to the 2019 stock data that has already been processed. You use AWS Step Functions to implement a solution that creates an Amazon EMR cluster and then terminates the cluster when it is no longer needed to conserve resources and reduce cost. You use AWS Lambda, Amazon Simple Notification Service (Amazon SNS), and state machines to achieve your goal.

You plan to use the following architecture to achieve your batch processing goal:

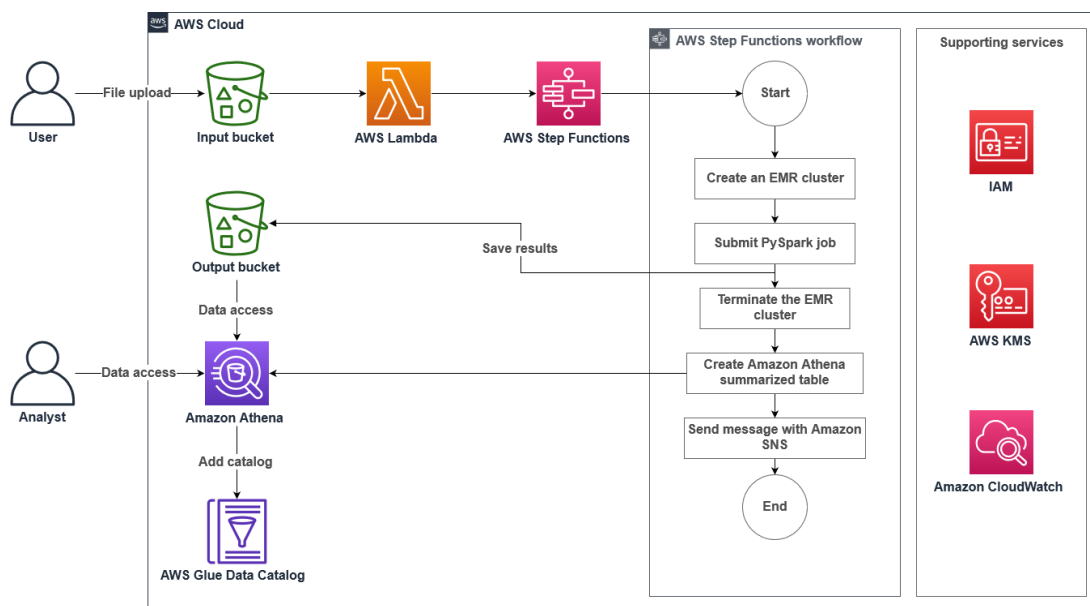


Image description: The preceding diagram depicts the various AWS services used in the lab architecture to achieve your batch processing goal.

Objectives

By the end of this lab, you should be able to do the following:

- Use Amazon Simple Storage Service (Amazon S3) Event Notifications and AWS Lambda to automate the batch processing of data.
- Use the Step Functions state machine language to:
 - Create an on-demand Amazon EMR cluster.

- Add an Apache Spark step job in Amazon EMR and create an Amazon Athena table to query the processed job.
 - Add an Amazon SNS topic to send a notification.
- Validate a Step Functions state machine run.
- Review an AWS Glue table and validate the processed data using Athena.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.

Caution: You must wait for the provisioned AWS services to be ready before you can continue.

2. To open the lab, choose **Open Console**.

You are automatically signed in to the AWS Management Console in a new web browser tab.

Warning: Do not change the **Region** unless instructed.

Common sign-in errors

Error: Choosing Start Lab has no effect

In some cases, certain pop-up or script blocker web browser extensions might prevent the **Start Lab** button from working as intended. If you experience an issue starting the lab:

- Add the lab domain name to your pop-up or script blocker's allow list or turn it off.
- Refresh the page and try again.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Task 1: Explore the lab environment

In this task, you review the account resources that were created when the lab was started. You review 2019 stock data, the Lambda function that starts the Step Functions step, and the Step Functions state machine.

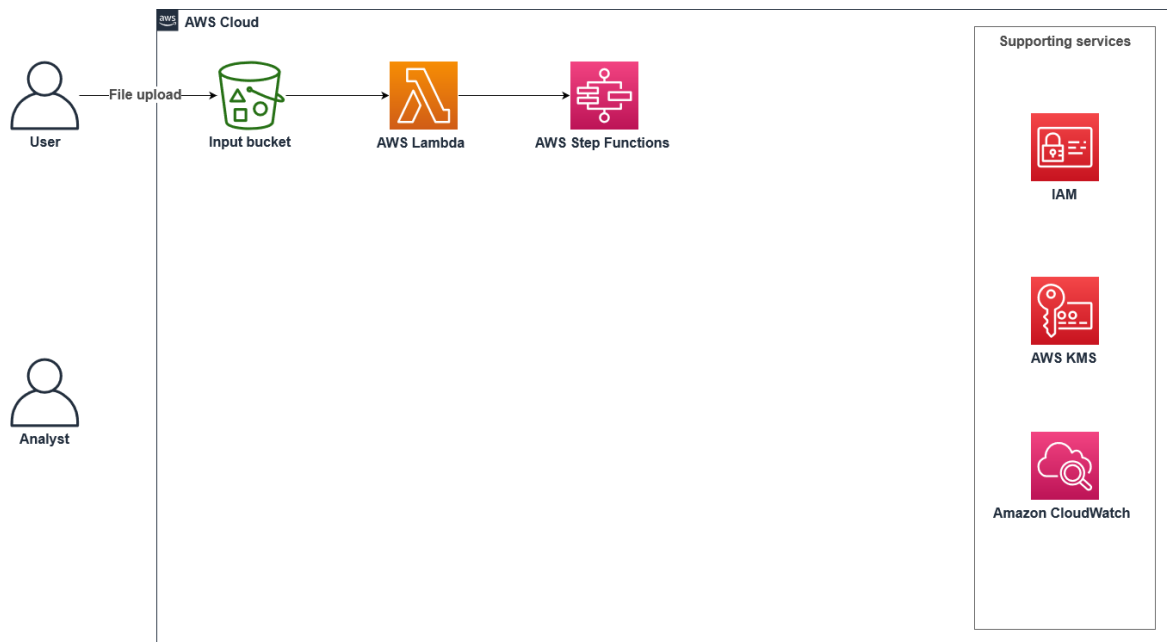


Image description: The preceding diagram depicts the account resources that were created when the lab was started.

Task 1.1: Review the contents in the Amazon Simple Storage Service (Amazon S3) bucket

- At the top of the AWS Management Console, in the search bar, search for and choose

S3

- Choose the bucket that starts with **databucket-**.

- The following folders were created for the task:

- data/** - stores the input stock price file in CSV format.
- logs/** - stores the Amazon EMR logs for troubleshooting any failures.
- output/** - stores the Spark-processed data in Parquet format.
- results/** - stores the Amazon Athena query results.
- scripts/** - stores the PySpark script to process data.

- Choose the link to open the **data/** folder.

A file called **stock_prices_2019.csv** is saved to the folder. This object lists stock prices for a variety of large tech companies (AAPL, SQ, AMZN, GE, M, TSLA, and MSFT) for the year 2019. Data columns you can find include *Trade_Date*, *Ticker*, *High*, *Low*, *Open*, *Close*, *Volume*, and *Adj_Close*.

File contents: The **stock_prices_2019.csv** file contains the following sample data:

Trade_Date	Ticker	High	Low	Open	Close	Volume	Adj_Close
2019-01-02	aapl	39.71250152	38.55749893	38.72249984	39.47999954	148158800	38.43973541
	msft	5878906	1884766	741211	223633	0	2597656

Trade Date	Ticker	High	Low	Open	Close	Volume	Adj_Close
2019-01-02	sq	57.83000183 105469	53.56000137 3291016	54.09999847 4121094	57.20000076 293945	13434000	57.20000076 293945
2019-01-02	amzn	1553.359985 3515625	1460.930053 7109375	1465.199951 171875	1539.130004 8828125	7983100	1539.130004 8828125
2019-01-02	ge	7.865385055 541992	7.125	7.173077106 47583	7.740385055 541992	134528264. 0	7.664851188 659668
2019-01-02	m	30.95999908 4472656	29.01000022 8881836	29.09000015 258789	30.76000022 8881836	8168200.0	27.27962875 366211
2019-01-02	tsla	63.02600097 65625	59.75999832 15332	61.22000122 0703125	62.0239982v 01.75	98.9400024 4140625	99.55000305 175781

Task 1.2: Review the Lambda function

- At the top of the AWS Management Console, in the search bar, search for and choose

Lambda

- On the **Functions** page, choose the **runFunction** lambda function.
- Scroll down to the **Code source** section and review the code from the **index.py** file.

Note: In this code, the Lambda function starts the *Step Functions* step if the event contains **databucket** as a source bucket and the event name contains **ObjectCreated:Put**.

Task 1.3: Review the Step Functions state machine definition

The Step Functions state machine definition was already created as a part of the lab build. In this task, you review the Step Functions state machine language. The state machine completes the following tasks:

- Creates an Amazon EMR cluster
- Runs Spark step jobs
- Tears down the Amazon EMR cluster resources
- Creates an Amazon Athena table to query the processed data
- Adds an Amazon SNS topic to notify the stakeholder that the task is complete

- At the top of the AWS Management Console, in the search bar, search for and choose

Step Functions

- On the **State machines** page, choose the state machine that starts with **BatchProcessingStep**.
- Choose the **Definition** tab.

The Step Functions state machine code is on the left, and the workflow diagram is on the right.

Knowledge check: Take a moment to review what the state machine does to automatically provision resources, process the data, and then terminate the resources to clean up the batch process. Look at the Amazon EMR cluster configurations and think about how the cluster can be optimized for cost and efficiency. Try to answer the following questions:

- What applications are selected for the Amazon EMR cluster?
- What kind of instances are used for the leader and core fleet?
- What Amazon SNS message are sent to the stakeholder when the steps are complete?

Answer: You can review an explanation of the Step Functions state machine code in the [Appendix section](#).

Task complete: You have successfully reviewed the account resources that were created when the lab was started.

Task 2: Run the Step Functions state machine task

Before you actually run the Step Functions steps, you need to complete some configurations. Once the configurations are set, you can upload the 2020 data and the batch processing should start. In this task, you complete those configuration steps in preparation for running the step function.

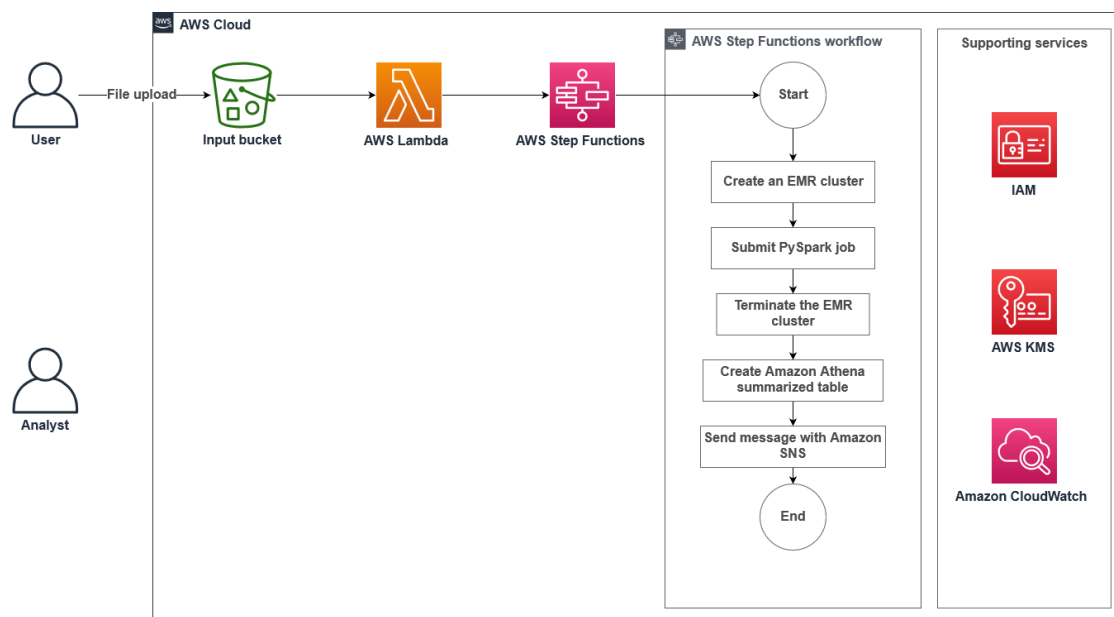


Image description: The preceding diagram depicts the various AWS services used in the lab architecture to achieve your batch processing goal.

Task 2.1: Configure S3 Event Notifications

In this task, you use the Amazon S3 Event Notifications feature to run the Lambda function when a new file is uploaded to the S3 folder. The Lambda function initiates the Step Functions run.

Learn more: Refer to *Amazon S3 Event Notifications* in the **Additional resources** section for more information.

13. At the top of the AWS Management Console, in the search bar, search for and choose

S3

14. Choose the bucket that starts with **databucket-**.

15. Choose the **Properties** tab and scroll down to the **Event notifications** section.

16. Choose **Create event notification**.

17. On the **Create event notification** page, in the **General configuration** section, configure the following:

- For **Event name**, enter

file_upload

- For **Prefix - optional**, enter

data/

- For **Suffix - optional**, enter

.CSV

18. In the **Event types** section, for **Object creation**, select **Put**.

19. In the **Destination** section:

- For **Destination**, select **Lambda function**.
- For **Specify Lambda function**, select **Choose from your Lambda functions**.
- For **Lambda function**, use the dropdown menu and select the **runFunction** lambda function.

20. Choose **Save changes**.

Task 2.2: Update and upload the PySpark script in the Amazon S3 dataBucket

Each night, your company receives the stock feed from various sources, but you want to identify high-volume trades so that you can serve a specific group of investors. This script identifies the trades with a volume greater than 100,000 shares. You need to add the script that completes this work to the scripts folder in your S3 bucket so that the state machine can complete the stock data processing.

21. **Copy edit:** Copy and paste the following script in a text editor of your choice.

Caution: Do not use MS Word to edit the code.

```
import sys
import time
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

bucket_name = "<dataBucket>"

spark = SparkSession.builder.appName("stock-
summary").getOrCreate()

stockDF =
spark.read.option("header", True).csv("s3://" + bucket_name + "/data/"
)

stockDF.registerTempTable("stock_data_view")

StockSummaryDF = spark.sql("SELECT `Trade_Date`, `Ticker`,
`Close` FROM stock_data_view WHERE Volume > 100000 ORDER BY Close
DESC")

StockSummaryDF.write.mode('overwrite').parquet("s3://" + bucket_nam
e + "/output/")

spark.stop()
```

22. Replace <dataBucket> with the **DataBucket** value shown to the left of these instructions.

23. Save the file as **script.py**.

24. At the top of the AWS Management Console, in the search bar, search for and choose

S3

.

25. Choose the bucket that starts with **databucket-**.

26. Choose the **scripts/** folder and then choose **Upload**.

27. On the **Upload** page, choose **Add files**.

28. Navigate to the folder in your local computer and select the **script.py** file you saved in the previous step.

29. Choose **Upload**.

Task 2.3: Update AWS Lake Formation permissions

As part of the Step Functions state machine, you use AWS Lake Formation to create a database in your AWS Glue Data Catalog. You need to grant the Step Functions state machine permission to create a database using AWS Identity and Access Management (IAM).

30. At the top of the AWS Management Console, in the search bar, search for and choose

`AWS Lake Formation`

.

31. If you are presented with a **Welcome to Lake Formation** pop-up window, configure below:

Note: If you see a “Failed to fetch catalogs” warning, you can safely ignore it.

- Select **Add myself**.
- Choose **Get started**.

32. On the **AWS Lake Formation** page, under **Administration** in the left pane, choose **Administrative roles and tasks**.

33. In the **Database creators** section, choose `Grant`.

34. On the **Grant permissions** pop-up window, configure the following:

- For **IAM users and roles**, use the dropdown menu and select the IAM role with - *stepFunctionRole*- in its name.

Note: You can also find the role by entering

`stepFunctionRole`

in the search box.

- Under **Create database**, select **Create database**.

35. Choose **Grant**.

Task 2.4: Subscribe to an Amazon SNS topic

As part of this lab build, an Amazon SNS topic was pre-created for you. In this task, you subscribe to the topic and confirm the subscription. This tells you when your batch processing job is complete.

36. At the top of the AWS Management Console, in the search bar, search for and choose

`Simple Notification Service`

.

Note: You may need to expand the left navigation pane by choosing the menu icon that is located on the left side of the console.

37. In the left pane, choose **Topics**.

38. Choose the topic with **-TaskCompleteSNS-** in its name.
39. In the **Subscriptions** tab, choose **Create subscription**.
40. On the **Create subscription** page, in the **Details** section, configure the following:
 - For **Protocol**, use the dropdown menu and select **Email**.
 - For **Endpoint**, enter an email address that can receive notifications from Amazon SNS.
41. Choose **Create subscription**.

You should receive an email to confirm your subscription.

42. Confirm the subscription by selecting the **Confirm subscription** link from the email.
43. In the left pane, choose **Topics**.
44. Choose the topic with **TaskCompleteSNS** in its name.

In the **Subscriptions** section, you should see your subscription status as **Confirmed**.

Task 2.5: Upload a file to the Amazon S3 bucket

In this task, you upload the 2020 stock data file to invoke the S3 event notification you created earlier, which runs a Lambda function to initiate a Step Functions step.

45. To download the [stock_prices_2020.csv](#) file, choose the text link.
46. At the top of the AWS Management Console, in the search bar, search for and choose  .
47. Choose the bucket that starts with **databucket-**.
48. Choose **data/** and choose **Upload**.
49. On the **Upload** page, choose .
50. Navigate to the folder in your local computer and choose the **stock_price_2020.csv** file you saved in a previous step.
51. Choose **Upload**.

Task complete: You have successfully completed the step function configuration steps and ran the Step Function state machine task.

Task 3: Validate the Step Functions run

In this task, you review and validate the Step Functions state machine code run status to ensure the batch processing is operating correctly. Once the run is complete, you review the AWS Glue table and run a query on Amazon Athena to validate the processed data.

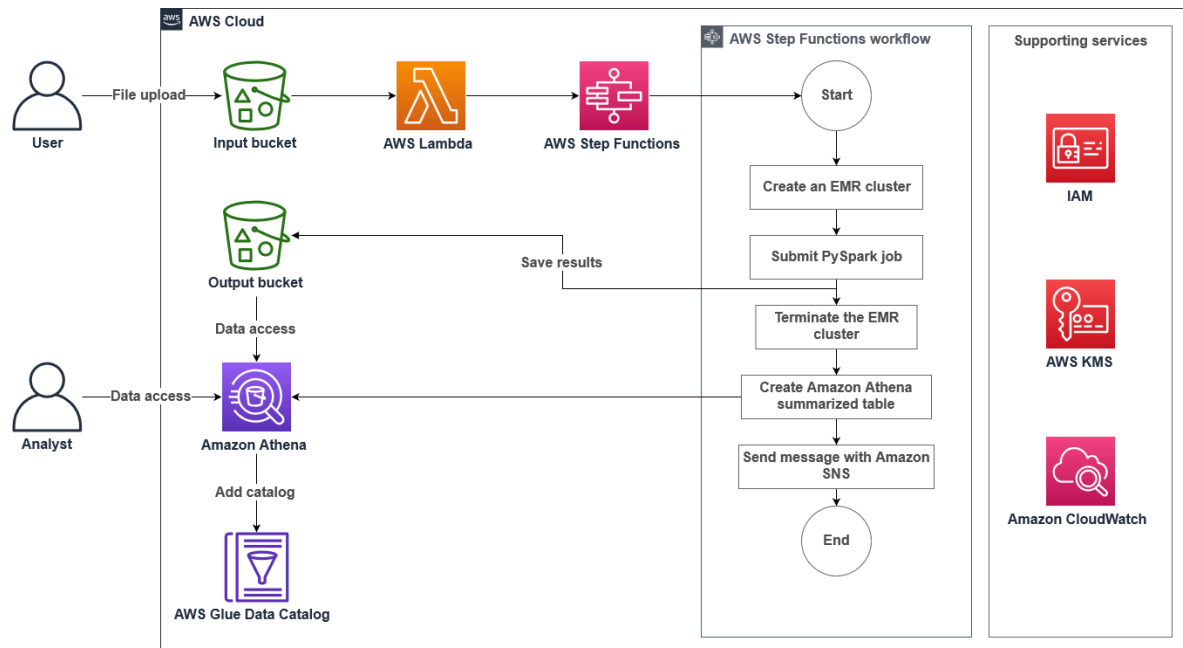


Image description: The preceding diagram depicts the various AWS services used in the lab architecture to achieve your batch processing goal.

Task 3.1: Validate Step Functions

52. At the top of the AWS Management Console, in the search bar, search for and choose

Step Functions

53. On the **State machines** page, choose the state machine that starts with **BatchProcessingStep-**.

54. In the **Executions** section, choose the name of the running state machine.

Note: If you do not see a state machine running, you might have to re-upload the sample file in the S3 bucket.

55. Choose the **Details** tab to view the state machine status. You should see the state machine in an **In progress** status.

Wait until all the steps are green and the state machine is in a **Succeeded** status. It takes approximately 15 to 20 minutes to complete the run.

After the task is complete, you should receive an email with the following confirmation message: **The Task is complete!**

Learn more: While you are waiting for the task to complete, refer to *Amazon States Language* in the **Additional resources** section for more information.

Task 3.2: Update Lake Formation permissions to view the table

Once you receive notification that the task is complete, you can grant yourself permissions to access the **stock_summary** table so you can view the processed data.

56. At the top of the AWS Management Console, in the search bar, search for and choose

AWS Lake Formation

57. In the left navigation pane, expand the **Administration** section and choose **Data Catalog settings**.

58. Clear both checkboxes for **Use only IAM access control for new databases** and **Use only IAM access control for new tables in new databases**.

Note: You can safely ignore the following error:

You don't have permissions to access this resource.

59. Choose **Save**.

60. In the left navigation pane, from the **Permissions** section, choose **Data permissions**.

61. Choose **Grant**.

Note: If you see a message that you are not authorized to perform lakeformation:DescribeLakeFormationIdentityCenterConfiguration, you can safely ignore it.

62. Configure:

- For **Principals**, choose **IAM users and roles**.
- For **IAM users and roles**, choose the Federated user you are logged in as.

Hint: Open the dropdown menu in the upper-right hand corner of your screen to learn the name of your *Federated user*. It may start with AWS Labs User.

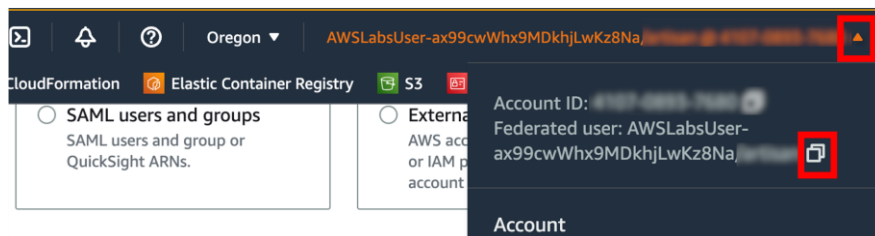


Image description: The preceding image shows how the Federated user can be copied from the AWS console.

- For **LF-Tags or catalog resources**, choose **Named data catalog resources**.
- For **Catalogs**, choose the **Default catalog**.
- For **Databases**, select **default** using the dropdown menu.
- For **Tables - optional**, select **stock_summary** using the dropdown menu.
- For **Table permissions**, choose **Select**.

Note: With these selections, you are allowing your user or role to perform *select* operations on the **stock_summary** table. The user or role can also grant *select* permissions to other users or roles.

63. Choose **Grant**.

Task 3.3: Validate the AWS Glue table and run a query on Athena to validate the data

In this task, you review the **stock_summary** table in the AWS Glue Data Catalog and run a query in Amazon Athena to validate the data. In AWS Glue, a *table* is the metadata definition that represents your data, including its schema.

64. At the top of the AWS Management Console, in the search bar, search for and choose

AWS Glue

65. In the left pane, expand the **Data catalog** section, and under **Databases** choose **Tables**.

66. On the **Tables** page, choose **stock_summary**.

You should see the schema and details of the table you created with Step Functions.

Note: If no tables appear, delete anything that is in the search box.

67. At the top of the AWS Management Console, in the search bar, search for and choose

Athena

Note: If you are brought to the **Amazon Athena** homepage, choose **Launch query editor** to navigate to the Query editor.

Note: If a **Workgroup primary settings** window opens, choose **Acknowledge**.

Before querying your data, you must first configure it for use as an Amazon Athena database.

68. In the **Data** section, configure the following:

- For **Data source**, use the dropdown menu and choose **AwsDataCatalog**.
- For **Database**, use the dropdown menu and choose **default**.

Under **Tables**, you can see the **stock_summary** table listed.

69. **Command:** Paste the following command in the query page:

```
SELECT * FROM "default"."stock_summary" ORDER BY trade_date DESC
LIMIT 10;
```

70. Choose **Run**.

Expected output: You should see 10 results in the **Results** section.

Note: If you receive a **No output location provided** error, use the following steps to update the query result location.

71. Choose **Settings** tab and then choose Manage.

72. On the **Manage settings** page:

- In the **Query result location and encryption** section, for **Location of query result - optional**, choose **Browse S3**. - -
- Select the radio button next to the **databucket-** S3 bucket.
- Select **Choose**.
- Choose **Save**

73. Return to the **Editor** tab and run the query again.

The output of the query will now be saved in the query result location that you just configured.

Task complete: You have successfully reviewed and validated the Step Functions state machine code run status and ran a query on Amazon Athena to validate the processed data.

Conclusion

You have successfully done the following:

- Used Amazon S3 Event Notifications and AWS Lambda to automate the batch processing of data.
- Reviewed the AWS Step Functions state machine language to:
 - Create an on-demand EMR cluster.
 - Add a Spark step job in Amazon EMR and create an Amazon Athena table to query the processed job.
 - Add an Amazon SNS topic to send a notification.
- Validated a Step Functions state machine run.
- Reviewed an AWS Glue table and validated the processed data using Athena.

End lab

Follow these steps to close the console and end your lab.

74. Return to the **AWS Management Console**.

75. At the upper-right corner of the page, choose **AWSLabsUser**, and then choose **Sign out**.

76. Choose End Lab and then confirm that you want to end your lab.

Additional resources

- [Amazon S3 Event Notifications](#)

- [Amazon States Language](#)

Appendix

Step Functions state machine code review

Let's review all five different states in the Step Functions state machine definition. The following section contains sample code. It is for reference only.

Create an EMR cluster

In this section, the example code is defining a few important features of the EMR cluster.

- **Resource:** Define the Create an EMR cluster task.
- **ReleaseLabel:** The release version of the EMR cluster.
- **Applications:** The list of applications to be installed.
- **ServiceRole:** IAM EMR Default Role name.
- **JobFlowRole:** IAM EC2 Instance Profile name.
- **LogUri:** Amazon S3 URI to store the EMR logs.
- **Instances:** Details of leader node and core nodes.
- **ResultPath:** Location to store the output of this state.
- **Next:** The name of the next state to be run.

```
*****  
***** EXAMPLE CODE *****  
*****
```

```
    "Create an EMR cluster": {  
      "Type": "Task",  
      "Resource":  
"arn:aws:states:::elasticmapreduce:createCluster.sync",  
      "Parameters": {  
        "Name": "labCluster",  
        "VisibleToAllUsers": true,  
        "ReleaseLabel": "emr-6.3.0",  
        "Applications": [  
          {  
            "Name": "Spark"  
          },  
          {  
            "Name": "Flink"  
          },  
          {  
            "Name": "Livy"  
          }  
        ],  
        "ServiceRole": "EMRDefaultRole",  
        "JobFlowRole": "EMREK2InstanceProfile",  
        "LogUri": "s3://lab3-databucket/logs/",  
        "Instances": {  
          "Ec2SubnetId": "subnet-015218b49b33dc30e",  
          "KeepJobFlowAliveWhenNoSteps": true,  
          "InstanceFleets": [  
            {  
              "Name": "MyLeaderFleet",
```

```

        "InstanceFleetType": "MASTER",
        "TargetOnDemandCapacity": 1,
        "InstanceTypeConfigs": [
            {
                "InstanceType": "m5.xlarge"
            }
        ]
    },
    {
        "Name": "MyCoreFleet",
        "InstanceFleetType": "CORE",
        "TargetOnDemandCapacity": 2,
        "InstanceTypeConfigs": [
            {
                "InstanceType": "m5.xlarge"
            }
        ]
    }
]
}
}
},
"ResultPath": "$.cluster",
"Next": "Submit PySpark Job"
}

```

Submit PySpark job

In this section, the example code is adding a step task to the EMR cluster.

- **Resource:** Define EMR and add step task.
- **ClusterId.\$:** The cluster ID value is retrieved from the previous result path.
- **HadoopJarStep:** This section contains the *spark-submit* command. In the following example, the Spark step is defined to run in cluster mode with run memory allocation of 1 GB. The location of the PySpark script is also defined here.
- **ResultPath:** Location to store the output of this state.
- **Next:** The name of the next state to be run.

```

*****
***** EXAMPLE CODE *****
*****

```

```

"Submit PySpark Job": {
    "Type": "Task",
    "Resource": "arn:aws:states:::elasticmapreduce:addStep.sync",
    "Parameters": {
        "ClusterId.$": "$.cluster.ClusterId",
        "Step": {
            "Name": "pyspark-job",
            "HadoopJarStep": {
                "Jar": "command-runner.jar",
                "Args": [
                    "spark-submit",
                    "--deploy-mode",
                    "cluster",
                    "--executor-memory",
                    "1g",
                    "s3://lab3-databucket/scripts/script.py"
                ]
            }
        }
    }
}

```

```

    ]
  }
},
"ResultPath": "$.sparkJob",
"Next": "Terminate Cluster"
}

```

Terminate the EMR cluster

In this section, the example code is adding a step task to terminate the EMR cluster.

- **Resource:** Define EMR decommission step task.
- **ClusterId.\$:** The cluster ID value is retrieved from the previous result path.
- **ResultPath:** Location to store the output of this state.
- **Next:** The name of the next state to be run.

```

*****
*****  EXAMPLE CODE  *****
*****

"Terminate the EMR Cluster": {
  "Type": "Task",
  "Resource": "arn:aws:states:::elasticmapreduce:terminateCluster",
  "Parameters": {
    "ClusterId.$": "$.cluster.ClusterId"
  },
  "ResultPath": "$.terminateCluster",
  "Next": "Create Amazon Athena Summarized Output Table"
}

```

Create a summarized output table in Athena

In this section, the example code is adding a step task to create an Athena table for the processed data in the Amazon S3 bucket.

- **Resource:** Define Athena *startQueryExecution* step task.
- **Parameters:** This section contains a *query* string, *WorkGroup* in Athena and a *result* path to save the query results.
- **ResultPath:** Location to store the output of this state.
- **Next:** The name of the next state to be run.

```

*****
*****  EXAMPLE CODE  *****
*****

"Create Amazon Athena summarized table": {
  "Type": "Task",
  "Resource": "arn:aws:states:::athena:startQueryExecution.sync",
  "Parameters": {
    "QueryString": "CREATE EXTERNAL TABLE IF NOT EXISTS
default.stock_summary(`Trade_Date` string,`Ticker` string,`Close`
string) ROW FORMAT SERDE
'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe' STORED AS
INPUTFORMAT
'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'

```

```

OUTPUTFORMAT
'org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat'
LOCATION 's3://lab3-databucket/output/' TBLPROPERTIES
('classification'='parquet', 'compressionType'='none',
'typeOfData'='file')",
    "WorkGroup": "primary",
    "ResultConfiguration": {
        "OutputLocation": "s3://lab3-databucket/results/"
    }
},
"ResultPath": "${athenaTable}",
"Next": "Send message to SNS"
}

```

Send completion message using Amazon SNS

In this section, the example code is adding a step task to send an Amazon SNS notification.

- **Resource:** Define SNS publish step task.
- **Parameters:** This section contains your SNS and message ARN.
- **End:** You define the value *true* to end the states.

```

*****
***** EXAMPLE CODE *****
*****

    "Send message with Amazon SNS": {
        "Type": "Task",
        "Resource": "arn:aws:states:::sns:publish",
        "Parameters": {
            "TopicArn": "arn:aws:sns:us-west-
2:1234567890:TaskCompleteAgain",
            "Message": {
                "Input": "The Task is complete!"
            }
        },
        "End": true
    }
}

```

To continue this lab, move on to [Task 2](#).

For more information about AWS Training and Certification, see <https://aws.amazon.com/training/>.

Your feedback is welcome and appreciated.

If you would like to share any feedback, suggestions, or corrections, please provide the details in our [AWS Training and Certification Contact Form](#).